CM3050 Mobile Development

Justin Ng Kwan Wei

ID 240662170

Mobile Application Name: Weather Forecast Application

Contents

Project Requirements 3

Project Research 4

Wireframing 5

User Feedback.....9

Development.....11

Unit Testing 19

Evaluation.....21

Project Requirements

To create a mobile application that incorporates the concepts covered in the course, I identified the following requirements for the project:

- API
- Local Storage
- Sensors
- Push Notification

Keeping the project requirements in mind, I began researching possible APIs that could be implemented into the application. After exploring the available options, I narrowed my focus to weather APIs, as they provide opportunities to incorporate API requests alongside other required features. I therefore decided to develop the application around Singapore's weather as it provides a familiar environment for developing and testing its features.

Before researching the available weather APIs, I identified the key features of the application and the requirements needed to support them.

Features	**Under**	**Requirements**
Retrieve Current Weather	API	Retrieve current weather conditions from the API
Weather Forecast	API	Retrieve past, present and future weather forecasts from the API
Current Location	Sensors	Retrieve the user's current location using the device's location services and display weather information for that location.
Bookmark Location	Local Storage	Allow users to bookmark locations and retain them locally
Set Notification	Push Notification	Allow users to receive weather-related push notifications

Table 1: Key Features Table

Based on these requirements, I began comparing available weather APIs to determine which would be most suitable for the application.

Project Research

1) API Selection

My initial research led me to the weather API provided by **data.gov.sg**. However, after exploring the API and its available data, I discovered several limitations, such as weather readings being associated with station IDs located around Singapore and the lack of future weather forecast data. As these limitations did not meet the requirements identified for the project, the API was not suitable for the application.

Following feedback from my lecturer, I explored **Open-Meteo** as an alternative weather API. In addition to not requiring an API key, Open-Meteo provides current, historical, and forecast weather data based on geographical coordinates. Therefore, Open-Meteo was selected as the weather API for the project as it provided the required forecast data while supporting location-based weather retrieval.

2) Weather Application Research

Research was also conducted on common features found in existing weather applications, such as Apple Weather and Google Weather. In addition to displaying weather forecasts based on the user's current location, these applications allow users to view weather conditions at other locations. This is useful for users who may want to check the weather at places they are travelling to or are interested in.

Another useful feature identified was the ability to bookmark or save locations. This allows users to quickly return to frequently viewed locations without having to search for them again.

These features were taken into consideration when developing the initial wireframes for the application. As a result, the application was designed to support the user's current location, browsing different locations around Singapore, and bookmarking locations for future access.

Wireframing

During the wireframing stage, I created several page designs to establish the application's layout, functionality, and navigation between different features. These wireframes provided an initial foundation for the application's design and development.

1) Home Page

The Home Page serves as the application's main screen and is designed to display weather information based on the user's current location. When the user starts the application, it will request permission to access the user's location. If permission is granted, the user's current latitude and longitude will be used to request the relevant weather information from the weather API.

In addition to the current weather, the API will also retrieve past and future weather forecasts. A dropdown list was initially designed to allow users to select a specific day and hour to view the corresponding weather forecast.

A weather reminder notification feature was also included on the Home Page. This allows users to set a reminder based on the weather at their current location.



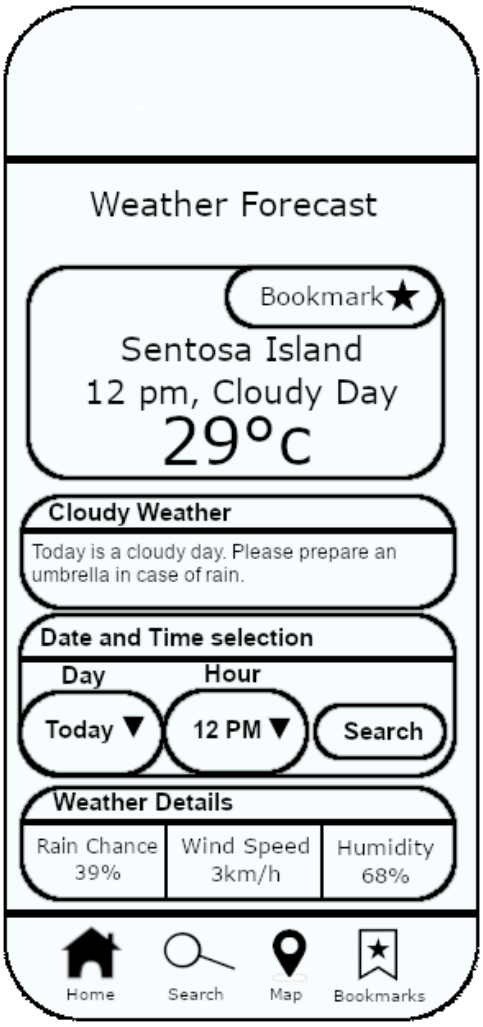
(Figure 2: Home Page)

2) Details Page

The Details Page is designed similarly to the Home Page but displays weather information for a selected location rather than the user's current location. Users can access this page by selecting a location from the Map, Search Results, or Bookmark Page.

As each location has predefined latitude and longitude coordinates, these values can be used to retrieve the relevant weather information. A bookmark toggle is also included, allowing users to save or remove the selected location from their bookmarks.

Unlike the Home Page, the Details Page does not include the weather reminder notification feature, as the reminder is intended to provide weather information based on the user's current location rather than a manually selected location.



(Figure 3: Details Page)

3.1) Search Page

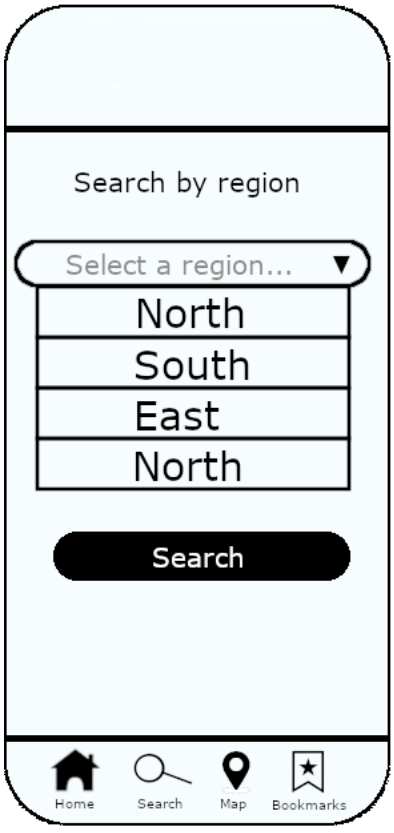
The Search Page allows users to search for locations in Singapore based on region. Users can select a region and tap the search button to view the locations available within the selected region on the Search Results Page.

Dividing locations by region provides users with a more structured way of browsing different areas of Singapore.

3.2) Search Results Page

The Search Results Page displays the locations available within the user's selected region. Users can select one of the displayed locations to access its Details Page and view the relevant weather information.

Each location is displayed as a selectable option to provide a simple transition between browsing for a location and viewing its detailed weather information.



(Figure 4: Search Page)



(Figure 5: Search Results Page)

4) Map Page

The Map Page is designed to display different locations around Singapore. It consists of a map of Singapore with key locations marked across the island, with each location's name and map coordinates being retrieved from a JSON file.

Users will be able to navigate the map using touchscreen controls by dragging vertically and horizontally. Upon selecting a marked location, the user will be directed to its Details Page to view more detailed weather information.

The Map Page provides an alternative to the Search Page by allowing users to visually explore locations based on where they are situated in Singapore.

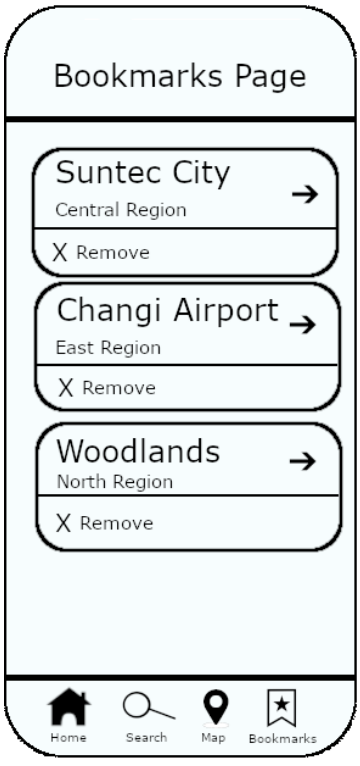
5) Bookmarks Page

The Bookmark Page is designed to display locations that the user has saved from the Details Page. Users can select a saved location to return to its Details Page or remove locations they no longer wish to keep bookmarked.

Each bookmarked location will also display its current temperature, allowing users to quickly view weather information without having to open the Details Page. This provides users with quicker access to locations they frequently check without having to search for the same location again.



(Figure 6: Map Page)



(Figure 7: Bookmarks Page)

User Feedback

After creating an initial prototype of the weather application, several students were invited to test its features and provide feedback on its usability and functionality. The feedback was reviewed to identify areas of the application that could be improved. The following changes were made based on the feedback received.

1)Usability Feedback

Feedback: Users felt that there should be an easier way to view weather information by hour and date without having to use dropdown lists on the Home and Details Pages.

Changes made: The dropdown lists were replaced with two horizontally scrollable selectors: one for selecting the hour and another for selecting the date. Each option was made directly selectable using touchable elements.

Result: This change allowed users to quickly switch between different hours and dates. Weather icons and temperatures were also implemented within each hourly selector, allowing users to preview upcoming weather conditions and temperatures at a glance without having to select each hour individually.

2)Navigation Feedback

Feedback: The Search Page required users to select a region before they could view the available locations on a separate Search Results Page. Users felt that this added an unnecessary step and screen transition. They suggested displaying all available locations directly, with the option to filter them by region.

Changes made: The Search Page was redesigned to display all available locations directly, while the previous region selection was changed into a filter. As the search results could now be displayed on the same page, the separate Search Results Page was removed. A similar filtering system was also introduced to the Bookmark Page to help users navigate through their saved locations.

Result: Removing the Search Results Page reduced the number of screen transitions required to browse locations. Users can now view all available locations immediately and use the region filter to narrow down the results when needed, creating a more direct and efficient search process.

3) Information Visibility Feedback

Feedback: Users felt that the Map Page did not provide enough weather information at a glance. Users had to manually select individual locations to check their weather conditions, which could be tedious when comparing multiple locations.

Changes made: The map markers were changed to display the current weather conditions for each location around Singapore.

Result: This change allows users to quickly compare the current weather conditions across different areas of Singapore directly from the Map Page, without having to select each location individually.

4) Error Handling Feedback

Feedback: During testing, users encountered several situations where the application was unable to retrieve the required information. These included denied location or notification permissions and the inability to retrieve weather information without an internet connection due to the application's reliance on the weather API.

Changes made: Error handling was introduced for situations where weather data could not be retrieved or when location and notification permissions were denied. Appropriate messages are displayed to inform users of the issue instead of leaving pages blank or features unresponsive.

Result: This provides users with clearer feedback when a feature is unavailable or an error occurs, allowing them to understand the cause of the issue rather than assuming that the application is not functioning correctly.

Development

1) Project Setup and Navigation

The application was developed using React Native and Expo.

The main application structure was setup in **App.js**, where navigation between the different screens is handled using React Navigation. Two types of navigation were used for this application: a Bottom Tab Navigator and a Stack Navigator.

The Bottom Tab Navigator appears at the bottom of the user's screen and allows users to navigate through the main screens used for the application: Home, Search. Map and Bookmarks. Meanwhile, the Stack Navigator is used for the Details Screen, which can be accessed by selecting a location from the Search, Map, or Bookmarks Screen.

The Home Screen is set as the initial screen displayed when the user opens the application.

2) Weather API Integration

The Open-Meteo Weather API requires latitude and longitude coordinates to retrieve weather information for a specific location.

Of the main pages in the application, all pages except the Search Page use weather information retrieved from the API. The Home Page uses the user's current location, while the Details Page relies on predefined coordinates based on the location selected by the user. The Map and Bookmarks Pages retrieve the current weather information for each of their listed locations.

A **locations.json** file was created to store information about each available location in Singapore. The latitude and longitude coordinates for these locations were obtained from OneMap and stored within the JSON file. For the Map Page, additional MapX and MapY coordinates were manually assigned to each location to determine the position of its marker on the Singapore map in the Map Page.

To retrieve weather information from **Open-Meteo**, two asynchronous functions were created using the JavaScript Fetch API and stored in **weatherapi.js**. The **getCurrentWeather()** function retrieves only the current temperature and weather condition, as it is used by the Map and Bookmarks Pages where less detailed weather information is required.

In comparison, **getWeatherForecast()** retrieves more detailed current and hourly weather information for the Home and Details Pages, including historical and forecast weather data.

```
// Used by Map and Bookmarks
export const getCurrentWeather = async (latitude, longitude) => {

  const url =
    `https://api.open-meteo.com/v1/forecast` +
    `?latitude=${latitude}` +
    `&longitude=${longitude}` +
    `&current=temperature_2m,weather_code` +
    `&timezone=auto`;

  const response = await fetch(url);

  if (!response.ok) {
    throw new Error('Unable to retrieve current weather');
  }

  return await response.json();
};

// Used by Home and Details
export const getWeatherForecast = async (latitude, longitude) => {
  const url =
    `https://api.open-meteo.com/v1/forecast` +
    `?latitude=${latitude}` +
    `&longitude=${longitude}` +
    `&current=temperature_2m,relative_humidity_2m,apparent_temperature,wind_speed_10m,weather_code` +
    `&hourly=temperature_2m,relative_humidity_2m,apparent_temperature,precipitation_probability,weather_code,wind_speed_10m` +
    `&past_days=3` +
    `&forecast_days=8` +
    `&timezone=auto`;

  const response = await fetch(url);

  if (!response.ok) {
    throw new Error('Unable to retrieve weather forecast');
  }

  return await response.json();
};
```

(Figure 8: fetch API functions)

Separating the API requests into two functions prevents the Map and Bookmarks Pages from requesting additional weather data that they do not require. The more detailed API request is only used on pages where users can view weather information across different dates and hours. In addition, the Map and Bookmarks Pages may make multiple API requests on a single page to retrieve the current weather for each listed location. Therefore, using the smaller **getCurrentWeather()** request avoids retrieving unnecessary historical and forecast data for every location.

3) Display of retrieved detailed weather information

The detailed forecast data retrieved using **getWeatherForecast()** is filtered based on the selected date and hour. The retrieved data includes the weather condition, temperature, humidity, rain chance, apparent temperature and wind speed. **getWeatherForecast()** is used by both the **Home Page** and **Details Page**.

For the Home Page, the user is prompted to grant permission for the application to access their current location. If permission is granted, the user's current latitude and longitude are used to retrieve the relevant weather forecast. If permission is denied, an error message is displayed to inform the user that their location cannot be accessed.

The retrieved weather information is displayed through several weather cards. The main weather card displays the selected date and time, location and temperature, followed by the current weather condition and weather advisory (Figure 10). Additional weather information, including humidity, rain chance, apparent temperature and wind speed, is displayed using four smaller cards (Figure 11).



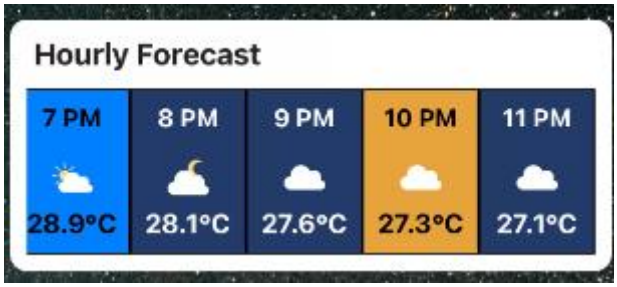
(Figure 10: Main Weather Card)



(Figure 11: Additional Weather Information)

To allow users to view weather conditions for different hours of the day, a horizontally scrollable selector is used to display the weather condition and temperature for each hour.

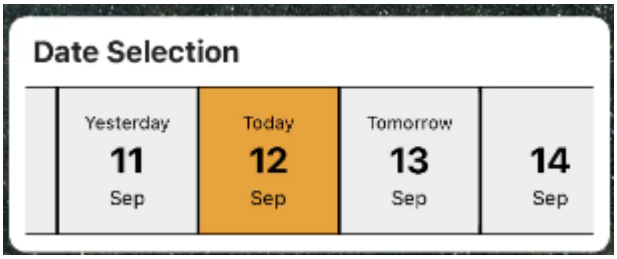
Each hourly option is implemented as a touchable component. When selected, the corresponding hourly card is highlighted to indicate the user's current selection, while the main weather information is updated to display the weather conditions for the selected hour.



(Figure 12: Hourly Forecast Horizontal Selector)

Similarly, to allow users to view weather conditions across different dates, a horizontally scrollable Date Selection selector is used to display the previous 3 days and the next 7 days. Each date is implemented as a touchable component. When a date is selected, the selected date is highlighted and the Hourly Forecast selector is updated to display the weather conditions available for that specific date.

The currently selected hour is also retained when switching between dates. For example, if the user is viewing the weather at 3 PM and selects another date, the application will automatically display the weather conditions at 3 PM for the newly selected date. The main weather information is then updated accordingly.



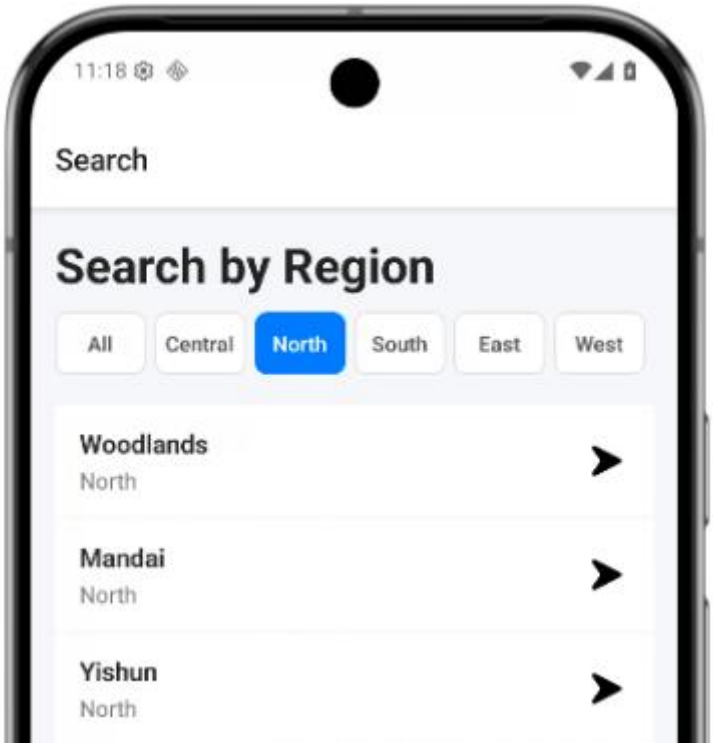
(Figure 13: Date Selection Horizontal Selector)

4) Search Page

To allow users to find weather information for different locations in Singapore, a **Search Page** was implemented. The available locations are organised according to their respective regions, allowing users to browse all locations or filter them based on a selected region.

The available location information is retrieved from **locations.json**. When a region is selected, the location data is filtered to display only locations that match the selected region. Selecting **All** removes the region filter and displays all available locations.

Each displayed location is implemented as a touchable component. When a location is selected, the user is navigated to its respective Details Page. The selected location's latitude and longitude are retrieved from **locations.json** and used to request its corresponding weather forecast from the API.



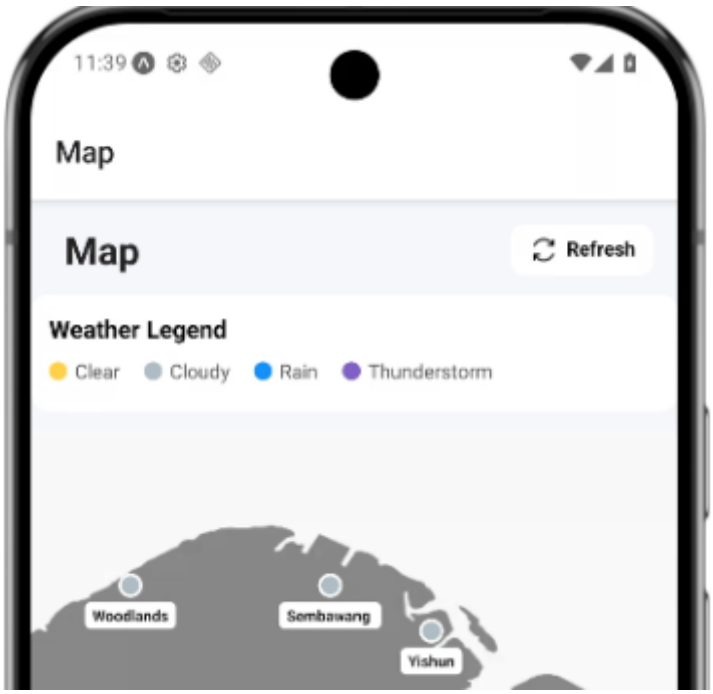
(Figure 14: Search Page)

5) Map Page

The Map Page provides an alternative way for users to browse weather locations visually. A map of Singapore is displayed using the ImageBackground component. As the map is larger than the available screen space, horizontal and vertical ScrollView components are used to allow users to navigate around different areas of the map.

locations.json was updated to include predefined mapX and mapY values for each location, allowing each location to be positioned as a point on the map. Each location point is implemented using TouchableOpacity, making it selectable. When a user selects a location, they are navigated to the corresponding Details Page, where the weather information for that location is retrieved and displayed.

The **getCurrentWeather()** function is also used to retrieve the current weather condition for each location using its stored latitude and longitude. The retrieved weather condition is represented by the colour of each location point, allowing users to compare the current weather conditions across different areas of Singapore at a glance. The user can also press the Refresh button to retrieve the latest weather readings for the displayed locations.



(Figure 15: Map Page)

6) Bookmark Implementation

To support bookmarks in the application, **AsyncStorage** is used to store bookmarked locations locally. This allows the application to retain the user's bookmarks even after the application is closed and reopened.

To allow users to add or remove bookmarks, a bookmark button is included on the Details Page. Depending on whether the selected location has already been bookmarked, the button displays either a bookmarked or unbookmarked state. Pressing the button adds or removes the location from the user's bookmark list.

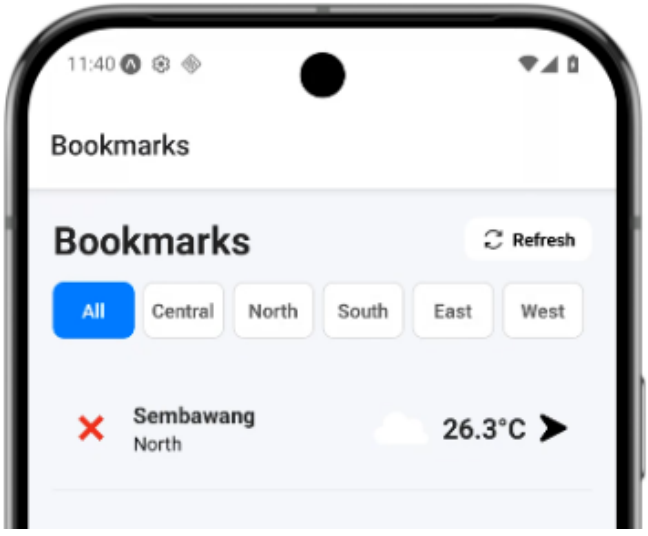
When the user navigates to the Bookmarks Page, all saved locations are retrieved from AsyncStorage and displayed. Similar to the Search Page, users can filter their bookmarked locations according to region.

The **getCurrentWeather()** function is used to retrieve the current weather condition and temperature for each bookmarked location using its stored latitude and longitude. Users can also press the Refresh button to retrieve the latest weather readings for their bookmarked locations.

Each bookmarked location is selectable and navigates the user to its corresponding Details Page, where more detailed weather information is displayed. A remove button is also provided beside each bookmarked location, allowing users to remove a location directly from the Bookmarks Page.



(Figure 15: Bookmark Button)



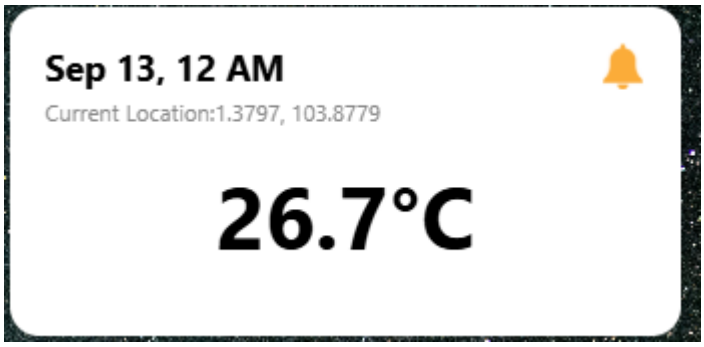
(Figure 16: Bookmarks Page)

7) Notification Implementation

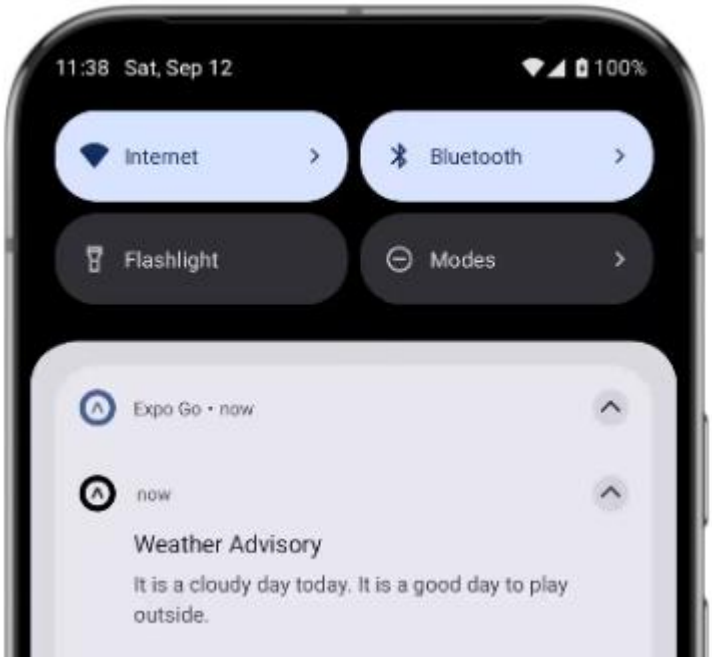
To support notifications in the application, the Expo Notifications library was imported using expo-notifications. This allows the application to request notification permission and schedule local notifications on the user's device.

The notification feature was implemented on the Home Page to provide users with a weather reminder based on their current location. When the notification button is selected, the application first requests permission to send notifications. If permission is granted, the application retrieves the weather information for the current date and hour and uses it to generate the weather advisory.

A local notification containing the weather advisory is then scheduled and displayed on the user's device. This allows users to receive the current weather advisory outside the application without having to continuously check the Home Page. If notification permission is denied, an alert is displayed to inform the user that notifications are disabled.



(Figure 17: Notification Button)



(Figure 18: Notification Alert)

Unit Testing

After user feedback and development, unit testing was conducted to ensure functionality of the application.

The unit tests will be conducted on the various components of the application such as navigation, API integration, local storage, sensors, and push notifications.

Test	Page	What to check	Expected result
Bottom Tab Navigation	All	Ensure the bottom tabs navigate to the correct pages	Home, Search, Map and Bookmark tabs should navigate to Home, Search Map and Bookmark pages respectively
Details Page Navigation	Map, Search, Bookmarks	User selects a location or location marker	Navigate to its appropriate details page.
Location permission denied	Home	User rejects location access	An error message should be displayed informing the user that location access was denied.
Notification Permission Denied	Home	User rejects notification access	An alert should be displayed informing the user that notification permission has been denied.
Push notification	Home	The user should be able to get a notification	Notification appears after a specified delay after clicking the notification button
No internet connection	Home, Details, Map, Bookmark	User has no internet access	An error message should be displayed when weather information cannot be retrieved.
Refresh	Home, Details, Map, Bookmarks	Ensure page is able to reload by pressing the refresh button	The latest weather information should be retrieved and displayed.
Current location	Home	Usage of location feature	Retrieval of user latitude and longitude coordinates.
Hour selection	Home, Details	User selects an hour in the time selector	Weather information updates to the selected hour
Date selection	Home, Details	User selects a date in the date selector	Weather information updates to the selected date. Time selector should also update to reflect the change in date.
Search Page Filter	Search	Ensure search page is able to filter by region	The search page should filter locations that belong to that region with the exception of ‘all’.

Map Scroll Navigation	Map	Ensure scrolling functionality works	User should be able to navigate the map by scrolling vertically or Horizontally.
Toggle Bookmark	Details	User taps the bookmark button to add Bookmark and tap again to remove it.	Bookmark button should be changed to reflect its bookmarked status. .
Bookmark persistence	Bookmark, Details	User closes or restarts the application.	Going to the previously bookmarked pages should still show it is still bookmarked. The Bookmarks Page should still show all the previously bookmarked locations.
Remove bookmarks from the Bookmark Page	Bookmark	Removal of a bookmark using the cross button.	The Bookmarked location would be removed from the Bookmarks page. Also check its Search page to see it is no longer bookmarked.

(Figure 19: Unit Testing Table)

Evaluation

After completing development and testing of the application, an evaluation was conducted to determine weather the final application successfully met the requirements established during the concept development stage. The evaluation considers the implementation of the main project requirements as well as the overall functionality of the application.

Requirement	Final Implementation	Evaluation
Weather API	Open-Meteo Weather Forecast API was implemented to retrieve historical and forecast weather information based on a provided set of latitude and longitude coordinates.	Met
Location Sensor	By enabling location permission in the application, the user's current location can be retrieved and display weather information in the Home page.	Met
View Weather Forecast	Users can view historical and forecast weather through horizontally scrolling hour and date selections.	Met
Location Browsing	Users can access different locations across Singapore through Search, Map and Bookmark Pages.	Met
Local Bookmark Storage	AsyncStorage was used to store bookmarked locations, allowing bookmarks to remain after closing or restarting the application.	Met
Set Notification	Expo Notification was implemented on the Home page. Pressing it will provide users with a local weather advisory notification based on the current weather.	Met
Error Handling	The application should have measures in place in the scenario of denied permissions or failure to retrieve weather data from the API.	Met

(Figure 20: Evaluation Table)

1)Strengths of the Final Application

Overall, the final application successfully achieved its main objective of allowing users to conveniently view weather information for different locations around Singapore. Users can view weather information based on their current location while also exploring other locations through the Search and Map Pages. The Bookmark feature further allows users to save frequently viewed locations for quicker access.

Changes made following user feedback also improved the overall usability of the application. The horizontally scrollable date and hourly selectors allow users to easily browse and compare weather conditions across different dates and times. Improvements to the Search Page provide a more direct way of browsing and filtering locations, while the weather markers on the Map Page allow users to compare current weather conditions across Singapore at a glance.

Error handling was also implemented to improve the reliability of the application. Situations such as denied location or notification permissions and failure to retrieve weather information are handled by providing appropriate feedback to the user.

2)Limitation and Possible Improvements

Despite meeting the main project requirements, several limitations remain.

Firstly, the application relies on an active internet connection to retrieve weather information from the Open-Meteo API. Without an internet connection, updated weather information cannot be retrieved or displayed. Future development could introduce offline storage, allowing recently retrieved weather data to be temporarily stored on the user's device and viewed when an internet connection is unavailable.

The locations available within the Search and Map Pages are currently limited to the predefined locations stored within locations.json. In addition, the Map Page uses a static image of Singapore with manually assigned mapX and mapY coordinates to position each location.

This approach requires new locations and their corresponding coordinates to be manually added to the application, making it difficult to expand the number of available locations. The manually assigned map coordinates may also not always accurately represent the geographical position of each location.

Future development could allow users to dynamically search for additional locations rather than relying only on predefined location data. An interactive mapping service could also be implemented to provide more accurate location positioning and improved map interactions.

Lastly, the current notification feature provides a weather advisory when manually triggered by the user. This could be further developed to automatically notify users when specific weather conditions are expected, such as heavy rain or thunderstorms.

3)Conclusion

Overall, the application successfully met the main requirements established for the project. The integration of weather data, location services, local storage and notifications resulted in a functional weather application that allows users to access and explore weather information across Singapore. While there are opportunities to further improve the application's map, location coverage and notification system, the final application provides the core functionality originally intended for the project.